Distributed Systems 1

Project Title: Decentralized Fault-Tolerant Group Chat System

How does this Project Work?

(Note: Passing an explicit integer ID like 100, 200, 300 as the second argument is optional, but very helpful during the live demo to clearly show the Bully Algorithm!)

Multi-Laptop Live Demo Setup (2+ Physical Machines)

Because eduroam blocks UDP broadcast traffic, connect all demo laptops to a **smartphone Wi-Fi hotspot** or personal router.

1. Detailed Code Breakdown & What Each File Achieves

File 1: config.py (Configuration & LAN Routing Helper)

- **What this code achieves:** Centralizes all protocol parameters (BROADCAST_PORT = 50000, HEARTBEAT_INTERVAL = 2.0, HEARTBEAT_TIMEOUT = 6.0, ELECTION_TIMEOUT = 2.0) so your team can tune timeouts in one place. It also includes get_local_ip(), which detects your laptop's real Wi-Fi/LAN IP address (e.g., 192.168.x.x) instead of 127.0.0.1 so nodes on different physical laptops can talk to each other during the demo.

```
"""
config.py - Shared Configuration & Protocol Constants
Decentralized Fault-Tolerant Group Chat System
"""

import socket

# Network Discovery Constants
BROADCAST_PORT = 50000
BROADCAST_IP = "<broadcast>"
BUFFER_SIZE = 4096

# Timing Constants (in seconds)
```

```
HEARTBEAT_INTERVAL = 2.0   # Send heartbeat every 2 seconds
HEARTBEAT_TIMEOUT = 6.0    # 3 missed heartbeats (6s) = Node considered crashed
ELECTION_TIMEOUT = 2.0     # Wait time for 'OK' responses in Bully Algorithm


def get_local_ip() -> str:
    """
    Determines the machine's active LAN IP address so peers across different
    physical machines can communicate via UDP unicast.
    """
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
    except Exception:
        ip = "127.0.0.1"
    finally:
        s.close()
    return ip
```

File 2: network_discovery.py (Dynamic Discovery & Group View)

- **What this code achieves:** Fulfills the **Dynamic Discovery of Hosts** requirement.
    1. Creates **two UDP sockets**: a **Broadcast socket** bound to port 50000 (SO_BROADCAST and SO_REUSEPORT enabled so you can also run multiple nodes on a single laptop) and a **Unicast socket** bound to an OS-assigned dynamic port.
    2. Maintains the thread-safe **Group View** dictionary (self.peers) mapping each node_id to its ip, port, username, and last_seen heartbeat timestamp.

3. When a node starts, broadcast_discovery() broadcasts a DISCOVER packet across the subnet. Every active peer receives it in listen_broadcast_loop(), adds the newcomer to its Group View, and replies via UDP unicast with DISCOVER_REPLY (sharing its own identity, the current Leader ID, and the latest delivered message sequence number).

```
"""
network_discovery.py - Module 1: Network & Dynamic Discovery Layer
Handles UDP Broadcast (Port 50000) for host discovery, UDP Unicast socket
communication, and maintains the shared Group View of active peers.
"""

import socket
import json
import time
import threading
from config import BROADCAST_PORT, BROADCAST_IP, BUFFER_SIZE,
get_local_ip


class NetworkDiscoveryManager:
    def __init__(self, node):
        self.node = node
        self.local_ip = get_local_ip()

        # 1. Create UDP Unicast Socket (OS assigns an available dynamic port)
        self.unicast_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.unicast_sock.bind(("0.0.0.0", 0))
        self.unicast_port = self.unicast_sock.getsockname()[1]

        # 2. Create UDP Broadcast Socket (Port 50000) for Dynamic Discovery
        self.bcast_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.bcast_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,
1)
```

```
        if hasattr(socket, "SO_REUSEPORT"):
            try:
                self.bcast_sock.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEPORT, 1)
            except OSError:
                pass
        self.bcast_sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST,
1)
        self.bcast_sock.bind(("", BROADCAST_PORT))

        # 3. Group View: maps node_id (int) -> dict of peer metadata
        self.view_lock = threading.Lock()
        self.peers = {
            self.node.node_id: {
                "ip": self.local_ip,
                "port": self.unicast_port,
                "username": self.node.username,
                "last_seen": time.time(),
            }
        }

    def send_unicast(self, target_ip: str, target_port: int, payload: dict):
        """Serializes a dictionary as JSON and sends it via UDP Unicast."""
        try:
            data = json.dumps(payload).encode("utf-8")
            self.unicast_sock.sendto(data, (target_ip, target_port))
        except Exception as e:
            print(f"[!] Unicast send error to {target_ip}:{target_port}: {e}")

    def send_broadcast(self, payload: dict):
        """Serializes a dictionary as JSON and broadcasts it on Port 50000."""
        try:
            data = json.dumps(payload).encode("utf-8")
            self.bcast_sock.sendto(data, (BROADCAST_IP, BROADCAST_PORT))
        except Exception:
```

```
            try:
                self.bcast_sock.sendto(data, ("255.255.255.255",
BROADCAST_PORT))
            except Exception as ex:
                print(f"[!] Broadcast send error: {ex}")

    def broadcast_discovery(self):
        """Broadcasts a DISCOVER packet when the node starts or joins the
network."""
        packet = {
            "type": "DISCOVER",
            "node_id": self.node.node_id,
            "username": self.node.username,
            "ip": self.local_ip,
            "port": self.unicast_port,
        }
        self.send_broadcast(packet)

    def update_peer(self, node_id: int, ip: str, port: int, username: str = "Peer") ->
bool:
        """Adds or updates a peer in the Group View. Returns True if newly
discovered."""
        node_id = int(node_id)
        with self.view_lock:
            is_new = node_id not in self.peers
            existing_name = self.peers.get(node_id, {}).get("username", username)
            self.peers[node_id] = {
                "ip": ip,
                "port": int(port),
                "username": username if username != "Peer" else existing_name,
                "last_seen": time.time(),
            }
            return is_new

    def remove_peer(self, node_id: int):
```

```
        """Removes a crashed peer from the Group View."""
        node_id = int(node_id)
        with self.view_lock:
            if node_id in self.peers and node_id != self.node.node_id:
                del self.peers[node_id]

    def get_peers_snapshot(self) -> dict:
        """Returns a thread-safe copy of the active Group View."""
        with self.view_lock:
            return dict(self.peers)

    def listen_broadcast_loop(self):
        """Daemon thread listening for UDP Broadcast DISCOVER packets on Port
50000."""
        while self.node.running:
            try:
                data, _ = self.bcast_sock.recvfrom(BUFFER_SIZE)
                msg = json.loads(data.decode("utf-8"))

                if msg.get("type") == "DISCOVER":
                    sender_id = int(msg["node_id"])
                    if sender_id == self.node.node_id:
                        continue

                    is_new = self.update_peer(
                        sender_id, msg["ip"], msg["port"], msg.get("username", "Peer")
                    )
                    if is_new:
                        print(
                            f"\n[Discovery] + Node {sender_id} ({msg.get('username')}) "
                            f"joined from {msg['ip']}:{msg['port']}"
                        )

                    # Reply via UDP Unicast with our identity, current Leader, and
sequence state
```

```
                reply = {
                    "type": "DISCOVER_REPLY",
                    "node_id": self.node.node_id,
                    "username": self.node.username,
                    "ip": self.local_ip,
                    "port": self.unicast_port,
                    "leader_id": self.node.election.leader_id,
                    "last_seq": self.node.multicast.last_delivered_seq,
                }
                self.send_unicast(msg["ip"], int(msg["port"]), reply)

                # If the newly joined node has a higher ID than the current leader, trigger election
                if (
                    self.node.election.leader_id is not None
                    and sender_id > self.node.election.leader_id
                ):
                    self.node.election.start_election()

            except Exception as e:
                if self.node.running:
                    print(f"[!] Broadcast listener error: {e}")

    def handle_discover_reply(self, msg: dict):
        """Processes a unicast DISCOVER_REPLY from an existing peer."""
        sender_id = int(msg["node_id"])
        is_new = self.update_peer(
            sender_id, msg["ip"], msg["port"], msg.get("username", "Peer")
        )
        if is_new:
            print(
                f"[Discovery] + Discovered existing peer: Node {sender_id} "
                f"({msg.get('username')}) at {msg['ip']}:{msg['port']}"
            )
```

```
        remote_leader = msg.get("leader_id")
        if remote_leader is not None:
            remote_leader = int(remote_leader)
            if remote_leader >= self.node.node_id:
                self.node.election.leader_id = remote_leader

        remote_seq = int(msg.get("last_seq", 0))
        if remote_seq > self.node.multicast.last_delivered_seq:
            self.node.multicast.last_delivered_seq = remote_seq
```

File 3: election.py (Leader Election via Bully Algorithm)

- **What this code achieves:** Fulfills the **Leader Election (Voting)** requirement.
    1. Uses each node's unique integer node_id to compare priorities (node_id > sender_id).
    2. When start_election() is called (at startup, when a higher-ID node joins, or when the Leader crashes), the node looks up all peers in the Group View with a higher ID and unicasts ELECTION packets to them.
    3. If no higher node replies with OK within 2.0 seconds (ELECTION_TIMEOUT), the node calls declare_victory(), synchronizes its message sequence counter with the highest delivered sequence number, and announces itself as COORDINATOR to all peers.

```
"""
election.py - Module 2: Voting / Leader Election Module
Implements the Bully Algorithm using comparable integer Node IDs.
The elected Leader serves as the central Message Sequencer for Total Ordering.
"""

import threading
from config import ELECTION_TIMEOUT
```

```
class BullyElectionManager:
    def __init__(self, node):
        self.node = node
        self.leader_id = None
        self.election_in_progress = False
        self.received_ok = False
        self.lock = threading.Lock()

    def start_election(self):
        """Initiates a Bully Algorithm election."""
        with self.lock:
            if self.election_in_progress:
                return
            self.election_in_progress = True
            self.received_ok = False

        print(f"\n[Election] Starting Bully Election (My Node ID:
{self.node.node_id})...")
        peers = self.node.discovery.get_peers_snapshot()

        # Find all active nodes in the Group View with a strictly higher ID
        higher_nodes = [
            (nid, info) for nid, info in peers.items() if nid > self.node.node_id
        ]

        if not higher_nodes:
            # No higher-ID nodes exist; immediately become Coordinator (Leader)
            self.declare_victory()
            return

        # Challenge all higher-ID nodes with an ELECTION message
        for nid, info in higher_nodes:
            self.node.discovery.send_unicast(
                info["ip"],
```

```
                info["port"],
                {
                    "type": "ELECTION",
                    "node_id": self.node.node_id,
                    "username": self.node.username,
                    "ip": self.node.discovery.local_ip,
                    "port": self.node.discovery.unicast_port,
                },
            )

        # Wait ELECTION_TIMEOUT seconds for any 'OK' response
        timer = threading.Timer(ELECTION_TIMEOUT,
self._evaluate_election_timeout)
        timer.daemon = True
        timer.start()

    def _evaluate_election_timeout(self):
        """Called after ELECTION_TIMEOUT to check if any higher node bullied
us."""
        with self.lock:
            got_ok = self.received_ok
            if got_ok:
                self.election_in_progress = False
                return

        self.declare_victory()

    def declare_victory(self):
        """Declares this node as the new Coordinator (Leader & Sequencer)."""
        with self.lock:
            self.leader_id = self.node.node_id
            self.election_in_progress = False

        # Synchronize the Leader's sequence counter with the highest delivered
sequence number
```

```
        self.node.multicast.sync_sequencer_on_leadership()

        print(
            f"\n[Election] *** I AM NOW THE ELECTED LEADER / SEQUENCER
"
            f"(Node {self.node.node_id}) ***"
        )

        # Broadcast COORDINATOR announcement to all peers in the Group View
        peers = self.node.discovery.get_peers_snapshot()
        for nid, info in peers.items():
            if nid != self.node.node_id:
                self.node.discovery.send_unicast(
                    info["ip"],
                    info["port"],
                    {
                        "type": "COORDINATOR",
                        "leader_id": self.node.node_id,
                        "username": self.node.username,
                        "ip": self.node.discovery.local_ip,
                        "port": self.node.discovery.unicast_port,
                    },
                )

    def handle_election_message(self, msg: dict):
        """Handles an incoming ELECTION message from a lower-ID node."""
        sender_id = int(msg["node_id"])
        self.node.discovery.update_peer(
            sender_id, msg["ip"], msg["port"], msg.get("username", "Peer")
        )

        if self.node.node_id > sender_id:
            # Reply with OK to bully the lower-ID sender and start our own election
            self.node.discovery.send_unicast(
                msg["ip"],
```

```
                int(msg["port"]),
                {"type": "OK", "node_id": self.node.node_id},
            )
            self.start_election()

    def handle_ok_message(self, msg: dict):
        """Handles an incoming OK message from a higher-ID node."""
        with self.lock:
            self.received_ok = True

    def handle_coordinator_message(self, msg: dict):
        """Handles an incoming COORDINATOR victory announcement."""
        new_leader = int(msg["leader_id"])
        self.node.discovery.update_peer(
            new_leader, msg["ip"], msg["port"], msg.get("username", "Peer")
        )

        if new_leader < self.node.node_id:
            self.start_election()
            return

        with self.lock:
            self.leader_id = new_leader
            self.election_in_progress = False

        print(
            f"\n[Election] *** Node {new_leader} ({msg.get('username', 'Leader')}) "
            f"is now the Elected Leader (Sequencer) ***"
        )
```

File 4: fault_tolerance.py (Heartbeats & Crash Recovery)

- **What this code achieves:** Fulfills the **Fault Tolerance** requirement.
    1. Runs a background thread (heartbeat_loop) that sends a UDP unicast HEARTBEAT packet to every peer in the Group View every **2 seconds**.
    2. Continuously inspects now - info["last_seen"] for every peer. If any peer is silent for **more than 6 seconds** (3 missed heartbeats), it is declared crashed and removed from the Group View.
    3. If the crashed node was the active **Leader**, fault_tolerance.py clears leader_id and immediately triggers self.node.election.start_election() so the chat recovers without human intervention.

```
"""
fault_tolerance.py - Module 3: Fault Tolerance & Heartbeat Module
Periodically sends UDP unicast HEARTBEAT packets to all peers in the Group View,
detects crashed regular peers or a crashed Leader, and triggers recovery.
"""

import time
from config import HEARTBEAT_INTERVAL, HEARTBEAT_TIMEOUT


class FaultToleranceManager:
    def __init__(self, node):
        self.node = node

    def heartbeat_loop(self):
        """
        Daemon thread that sends periodic heartbeats every 2 seconds and checks
        if any peer has exceeded the 6-second timeout (3 missed heartbeats).
        """
        while self.node.running:
            time.sleep(HEARTBEAT_INTERVAL)
```

```
        now = time.time()

        # Refresh our own timestamp in the Group View
        self.node.discovery.update_peer(
            self.node.node_id,
            self.node.discovery.local_ip,
            self.node.discovery.unicast_port,
            self.node.username,
        )

        peers = self.node.discovery.get_peers_snapshot()

        # 1. Send UDP Unicast HEARTBEAT to all other active peers
        for nid, info in peers.items():
            if nid != self.node.node_id:
                self.node.discovery.send_unicast(
                    info["ip"],
                    info["port"],
                    {
                        "type": "HEARTBEAT",
                        "node_id": self.node.node_id,
                        "username": self.node.username,
                        "ip": self.node.discovery.local_ip,
                        "port": self.node.discovery.unicast_port,
                    },
                )

        # 2. Check for crashed nodes (missed 3 consecutive heartbeats = 6.0s)
        leader_crashed = False
        for nid, info in peers.items():
            if nid != self.node.node_id and (now - info["last_seen"]) >
HEARTBEAT_TIMEOUT:
                print(
                    f"\n[Fault Tolerance] - Node {nid} ({info.get('username')}) "
```

```
                    f"timed out ({HEARTBEAT_TIMEOUT}s). Removing from Group
View."
                )
                self.node.discovery.remove_peer(nid)
                if nid == self.node.election.leader_id:
                    leader_crashed = True

        # 3. If the Leader crashed, immediately trigger a Bully Election
        if leader_crashed:
            print(
                "[Fault Tolerance] ! Elected Leader crashed! "
                "Triggering Bully Leader Election..."
            )
            self.node.election.leader_id = None
            self.node.election.start_election()

    def handle_heartbeat(self, msg: dict):
        """Updates the last_seen timestamp when a HEARTBEAT arrives from a
peer."""
        sender_id = int(msg["node_id"])
        is_new = self.node.discovery.update_peer(
            sender_id, msg["ip"], msg["port"], msg.get("username", "Peer")
        )
        if is_new:
            print(
                f"\n[Discovery] + Added Node {sender_id} ({msg.get('username')}) "
                f"via Heartbeat from {msg['ip']}:{msg['port']}"
            )
```

File 5: multicast.py (Sequencer-Based Reliable Total Order Multicast)

- **What this code achieves:** Fulfills the **Reliable Ordered Multicast** requirement.

1. **Sequencing (handle_chat_request_as_leader)**: When any user sends a message, their node sends CHAT_REQ to the Leader. The Leader increments self.seq_counter (1, 2, 3...), stores the packet in self.message_history, and unicasts CHAT_ORDERED(seq) to all peers in the Group View.
2. **Strict Total Order & Hold-Back Queue (handle_ordered_message)**: Every peer tracks self.last_delivered_seq.
    - If seq == last_delivered_seq + 1, the node prints the message immediately and checks if last_delivered_seq + 1 is waiting in self.hold_back_queue.
    - If seq > last_delivered_seq + 1 (meaning an earlier message was dropped in transit), the node **does not print the out-of-order message yet**. Instead, it places it into self.hold_back_queue[seq] and sends unicast NACK packets for every missing sequence number (range(last_delivered_seq + 1, seq)).
3. **NACK Recovery (handle_nack)**: When the Leader (or any peer) receives a NACK, it looks up missing_seq in self.message_history and retransmits it directly to the requester, which unblocks the Hold-Back Queue.
4. **Built-in Demo Packet Drop Simulator (simulate_drop_next)**: Because local networks rarely drop UDP packets on command during a 15-minute presentation, typing /sim_drop in the terminal arms a one-time artificial drop of the next incoming message so you can visibly prove to the examiner that your Hold-Back Queue and NACK retransmission work in real time!

```
"""
multicast.py - Module 4: Reliable Ordered Multicast Module
Implements Sequencer-Based Total Order Multicast with a Hold-Back Queue
and Negative Acknowledgements (NACKs) for packet loss recovery.
"""

import threading
```

```
class ReliableMulticastManager:
    def __init__(self, node):
        self.node = node
        self.lock = threading.Lock()

        # Sequencer counter (used only when this node is the Elected Leader)
        self.seq_counter = 0

        # Total order delivery state (tracked by every node)
        self.last_delivered_seq = 0
        self.hold_back_queue = {}   # {seq_num: msg_dict} for out-of-order
messages
        self.message_history = {}   # {seq_num: msg_dict} cache for NACK
retransmissions

        # Demo feature: allows simulating a dropped UDP packet to prove Hold-Back
Queue & NACKs
        self.simulate_drop_next = False

    def sync_sequencer_on_leadership(self):
        """
        Ensures a newly elected Leader continues assigning sequence numbers
seamlessly
        from the highest known sequence number in the chat room.
        """
        with self.lock:
            highest_cached = max(self.message_history.keys(), default=0)
            self.seq_counter = max(
                self.seq_counter, self.last_delivered_seq, highest_cached
            )

    def send_chat_request(self, text: str):
        """
```

```
        Called when the local user types a message. Unicasts CHAT_REQ to the
Leader
        (Sequencer) so it can assign a global sequence number.
        """
        leader_id = self.node.election.leader_id
        if leader_id is None:
            print("[Multicast] ! No Leader elected yet. Hold on while election
completes...")
            self.node.election.start_election()
            return

        peers = self.node.discovery.get_peers_snapshot()
        if leader_id not in peers:
            print("[Multicast] ! Leader not found in Group View. Starting election...")
            self.node.election.start_election()
            return

        leader_info = peers[leader_id]
        self.node.discovery.send_unicast(
            leader_info["ip"],
            leader_info["port"],
            {
                "type": "CHAT_REQ",
                "sender_id": self.node.node_id,
                "sender": self.node.username,
                "text": text,
            },
        )

    def handle_chat_request_as_leader(self, msg: dict):
        """
        Executed ONLY by the Elected Leader (Sequencer).
        Increments the global sequence counter, caches the message, and multicasts
        CHAT_ORDERED to every peer in the Group View.
        """
```

```
        if self.node.election.leader_id != self.node.node_id:
            return

        with self.lock:
            self.seq_counter += 1
            seq = self.seq_counter
            ordered_packet = {
                "type": "CHAT_ORDERED",
                "seq": seq,
                "sender_id": msg.get("sender_id"),
                "sender": msg["sender"],
                "text": msg["text"],
            }
            self.message_history[seq] = ordered_packet

        peers = self.node.discovery.get_peers_snapshot()
        for nid, info in peers.items():
            self.node.discovery.send_unicast(info["ip"], info["port"], ordered_packet)

    def handle_ordered_message(self, msg: dict):
        """
        Processes an incoming CHAT_ORDERED packet.
        - Delivers immediately if seq == last_delivered_seq + 1
        - Buffers in Hold-Back Queue and sends NACK if seq > last_delivered_seq +
1
        """
        seq = int(msg["seq"])

        with self.lock:
            # Demo helper: optionally drop one incoming packet to demonstrate NACK
recovery
            if self.simulate_drop_next and not msg.get("retransmitted", False):
                self.simulate_drop_next = False
                print(
                    f"\n[Demo Simulation] ! Artificially dropped incoming packet "
```

```
                f"seq={seq} ('{msg['text']}') to demonstrate Hold-Back Queue!"
            )
            return

        # Cache message for peer/leader retransmission support
        self.message_history[seq] = msg

        # Case 1: Duplicate or already delivered message
        if seq <= self.last_delivered_seq:
            return

        # Case 2: Exact next expected sequence number -> Deliver immediately!
        if seq == self.last_delivered_seq + 1:
            self._deliver_to_user(msg)
            self.last_delivered_seq = seq

            # Drain any consecutive messages waiting in the Hold-Back Queue
            while (self.last_delivered_seq + 1) in self.hold_back_queue:
                next_seq = self.last_delivered_seq + 1
                buffered_msg = self.hold_back_queue.pop(next_seq)
                print(
                    f"[Hold-Back Queue] -> Unblocking buffered message
seq={next_seq}"
                )
                self._deliver_to_user(buffered_msg)
                self.last_delivered_seq = next_seq

        # Case 3: Sequence gap detected (seq > last_delivered_seq + 1)
        elif seq > self.last_delivered_seq + 1:
            self.hold_back_queue[seq] = msg
            missing_list = list(range(self.last_delivered_seq + 1, seq))
            print(
                f"\n[Multicast] ! Sequence gap detected! Received seq={seq}, "
                f"expected {self.last_delivered_seq + 1}. "
                f"Buffered seq={seq} in Hold-Back Queue. "
```

```
                f"Sending NACK for missing seq: {missing_list}"
            )
            self._send_nacks(missing_list)

    def _deliver_to_user(self, msg: dict):
        """Prints the delivered message in strict global sequence order."""
        tag = " (recovered via NACK)" if msg.get("retransmitted") else ""
        print(f"\n[Chat #{msg['seq']}] {msg['sender']}: {msg['text']}{tag}")

    def _send_nacks(self, missing_seqs: list):
        """Sends unicast NACK requests to the Leader (and peers) for missing
sequence numbers."""
        peers = self.node.discovery.get_peers_snapshot()
        leader_id = self.node.election.leader_id

        targets = (
            [peers[leader_id]]
            if leader_id in peers
            else [info for nid, info in peers.items() if nid != self.node.node_id]
        )

        for missing_seq in missing_seqs:
            nack_packet = {
                "type": "NACK",
                "missing_seq": missing_seq,
                "requester_ip": self.node.discovery.local_ip,
                "requester_port": self.node.discovery.unicast_port,
            }
            for target in targets:
                self.node.discovery.send_unicast(
                    target["ip"], target["port"], nack_packet
                )

    def handle_nack(self, msg: dict):
        """
```

```
        Responds to a peer's NACK by retransmitting the requested message from
        our message_history cache.
        """
        missing_seq = int(msg["missing_seq"])
        with self.lock:
            cached_msg = self.message_history.get(missing_seq)

        if cached_msg:
            retransmit_copy = dict(cached_msg)
            retransmit_copy["retransmitted"] = True
            print(
                f"\n[Multicast] Retransmitting cached message seq={missing_seq} "
                f"to {msg['requester_ip']}:{msg['requester_port']} in response to
NACK."
            )
            self.node.discovery.send_unicast(
                msg["requester_ip"], int(msg["requester_port"]), retransmit_copy
            )
```

File 6: main.py (Symmetric Node Runner & Interactive CLI)

- **What this code achieves:** Fulfills the **Symmetric Peer-to-Peer & Multithreading Architectural Model**. It instantiates all four managers, spawns the three background daemon threads (listen_broadcast_loop, listen_unicast_loop, and heartbeat_loop), triggers initial peer discovery and leader election, and runs an interactive command-line loop supporting live inspection commands (/peers, /leader, /sim_drop, /queue, /history, /elect, /help, /quit).

```
"""
main.py - Main Entry Point & CLI User Interface Layer
Ties together Dynamic Discovery, Bully Leader Election, Heartbeat Fault
Tolerance,
and Sequencer-Based Reliable Ordered Multicast into a symmetric Peer Node.
"""
```

```
import sys
import time
import json
import random
import threading
from config import BUFFER_SIZE
from network_discovery import NetworkDiscoveryManager
from election import BullyElectionManager
from fault_tolerance import FaultToleranceManager
from multicast import ReliableMulticastManager


class ChatNode:
    def __init__(self, username: str, custom_id: int = None):
        self.username = username
        self.running = True

        # Generate a unique comparable integer ID (millisecond timestamp suffix +
random)
        if custom_id is not None:
            self.node_id = int(custom_id)
        else:
            self.node_id = (int(time.time() * 1000) % 1000000) + random.randint(10,
999)

        # Initialize the 4 core Distributed Systems modules
        self.discovery = NetworkDiscoveryManager(self)
        self.election = BullyElectionManager(self)
        self.fault_tolerance = FaultToleranceManager(self)
        self.multicast = ReliableMulticastManager(self)

        print("=" * 72)
        print(f" Decentralized Fault-Tolerant Group Chat Node Started")
        print(f" Username : {self.username}")
```

```
        print(f" Node ID  : {self.node_id} (Used for Bully Leader Election)")
        print(f" Address  : {self.discovery.local_ip}:{self.discovery.unicast_port}")
        print(" Commands : Type any message to chat, or use /peers, /leader,")
        print("           /sim_drop, /queue, /history, /elect, /help, /quit")
        print("=" * 72)

    def listen_unicast_loop(self):
        """
        Daemon thread that receives all UDP Unicast protocol packets and dispatches
        them to the appropriate module.
        """
        while self.running:
            try:
                data, _ = self.discovery.unicast_sock.recvfrom(BUFFER_SIZE)
                msg = json.loads(data.decode("utf-8"))
                mtype = msg.get("type")

                if mtype == "DISCOVER_REPLY":
                    self.discovery.handle_discover_reply(msg)
                elif mtype == "HEARTBEAT":
                    self.fault_tolerance.handle_heartbeat(msg)
                elif mtype == "ELECTION":
                    self.election.handle_election_message(msg)
                elif mtype == "OK":
                    self.election.handle_ok_message(msg)
                elif mtype == "COORDINATOR":
                    self.election.handle_coordinator_message(msg)
                elif mtype == "CHAT_REQ":
                    self.multicast.handle_chat_request_as_leader(msg)
                elif mtype == "CHAT_ORDERED":
                    self.multicast.handle_ordered_message(msg)
                elif mtype == "NACK":
                    self.multicast.handle_nack(msg)

            except Exception as e:
```

```
            if self.running:
                print(f"[!] Unicast listener error: {e}")

    def start(self):
        """Starts background daemon threads, runs discovery, and enters the CLI
loop."""
        threading.Thread(
            target=self.discovery.listen_broadcast_loop, daemon=True
        ).start()
        threading.Thread(target=self.listen_unicast_loop, daemon=True).start()
        threading.Thread(
            target=self.fault_tolerance.heartbeat_loop, daemon=True
        ).start()

        # Broadcast DISCOVER packet to find active peers on the local network
        self.discovery.broadcast_discovery()
        time.sleep(1.2)

        # If no Leader was discovered or our ID is higher, initiate Bully Election
        if (
            self.election.leader_id is None
            or self.node_id > self.election.leader_id
        ):
            self.election.start_election()

        self.run_cli()

    def run_cli(self):
        """Main thread interactive command-line interface."""
        while self.running:
            try:
                user_input = input().strip()
                if not user_input:
                    continue
```

```
            if user_input.startswith("/"):
                self.handle_slash_command(user_input)
            else:
                self.multicast.send_chat_request(user_input)

        except (KeyboardInterrupt, EOFError):
            print("\n[*] Shutting down node...")
            self.running = False
            break

    def handle_slash_command(self, cmd: str):
        """Built-in helper commands to inspect state during the live demo."""
        if cmd == "/peers":
            peers = self.discovery.get_peers_snapshot()
            print("\n--- Active Group View (Discovered Peers) ---")
            for nid, info in sorted(peers.items()):
                role = " [LEADER / SEQUENCER]" if nid == self.election.leader_id
else ""
                self_tag = " (Me)" if nid == self.node_id else ""
                print(
                    f"  • Node {nid}: {info['username']} @ {info['ip']}:{info['port']}"
                    f"{role}{self_tag}"
                )
            print("-----------------------------------------")

        elif cmd == "/leader":
            lid = self.election.leader_id
            peers = self.discovery.get_peers_snapshot()
            lname = peers.get(lid, {}).get("username", "Unknown")
            print(f"\n[Status] Current Elected Leader: Node {lid} ({lname})")

        elif cmd == "/elect":
            self.election.start_election()

        elif cmd == "/sim_drop":
```

```
            self.multicast.simulate_drop_next = True
            print(
                "\n[Demo Simulation] Armed! The NEXT incoming ordered chat message "
                "will be artificially dropped so you can demonstrate the Hold-Back "
                "Queue and automatic NACK recovery when the subsequent message arrives."
            )

        elif cmd == "/queue":
            print(
                f"\n[Multicast State] Last Delivered Seq: {self.multicast.last_delivered_seq} "
                f"| Hold-Back Queue: {list(self.multicast.hold_back_queue.keys())}"
            )

        elif cmd == "/history":
            print("\n--- Delivered Message History Cache ---")
            for seq, m in sorted(self.multicast.message_history.items()):
                print(f"  [{seq}] {m['sender']}: {m['text']}")
            print("---------------------------------------")

        elif cmd == "/help":
            print(
                "\nAvailable Demo Commands:\n"
                "  /peers    - Show all active nodes in the Group View\n"
                "  /leader   - Show the current Elected Leader (Sequencer)\n"
                "  /sim_drop - Drop the next incoming chat packet to demo Hold-Back Queue & NACK\n"
                "  /queue    - Show last delivered seq and Hold-Back Queue contents\n"
                "  /history  - Show cached messages available for NACK retransmission\n"
                "  /elect    - Manually trigger a Bully Leader Election\n"
                "  /quit     - Exit the chat node"
            )
```

```
        elif cmd == "/quit":
            print("[*] Exiting chat node...")
            self.running = False
            sys.exit(0)
        else:
            print(f"[!] Unknown command '{cmd}'. Type /help for options.")


if __name__ == "__main__":
    username_arg = sys.argv[1] if len(sys.argv) > 1 else f"User-{random.randint(10,
99)}"
    custom_id_arg = int(sys.argv[2]) if len(sys.argv) > 2 else None
    node = ChatNode(username=username_arg, custom_id=custom_id_arg)
    node.start()
```

3. Step-by-Step Instructions to Run & Demo the Project

Step 1: Extract and Open Terminal

Download Decentralized_Group_Chat_Project.zip above, unzip it on your MacBook Air, and open your terminal inside the folder:

```
cd decentralized_chat_project
```

(No pip install is needed because the project uses 100% standard Python 3 libraries).

Step 2: Test Locally on Your Laptop (3 Terminal Tabs)

Open **3 terminal tabs** and launch three peer nodes. You can let the script generate a timestamp-based ID automatically (python3 main.py Alice), or pass a clean number (100, 200, 300) as the second argument so it is effortless to follow during your presentation:

Terminal 1:

```
python3 main.py Alice 100
```

-

Terminal 2:

```
python3 main.py Bob 200
```

-

Terminal 3:

```
python3 main.py Charlie 300
```

Step 3: Step-by-Step Script for Your 15-Minute Live Demo

During your 15-minute assessed demonstration, the lecturer will ask you to run the system across **at least two physical machines** and demonstrate each project requirement live.

> **Important Network Note:** Connect your laptops to a **smartphone Wi-Fi hotspot** before your slot starts, as eduroam blocks UDP broadcast traffic.

1. **Demonstrate Dynamic Discovery (network_discovery.py)**:
    - Start Alice (ID 100) on Laptop 1 and Bob (ID 200) on Laptop 2.
    - Point out the [Discovery] + Node 200 (Bob) joined... log on Alice's screen and run /peers to show the populated Group View without any hardcoded IP addresses.
2. **Demonstrate Bully Leader Election (election.py)**:
    - Point out that Bob (200) automatically bullied Alice (100) and became the Coordinator.
    - Now launch Charlie (300) on either laptop. Show how Charlie is dynamically discovered, challenges Bob, and announces [Election] *** Node 300 (Charlie) is now the Elected Leader (Sequencer) ***.
3. **Demonstrate Total Order Multicast & NACK Recovery (multicast.py)**:
    - Send a couple of normal chat messages from different terminals and show [Chat #1] and [Chat #2] appearing in identical global order everywhere.
    - Now, type /sim_drop on Alice (Node 100).
    - Have Charlie send "Message Three" (#3). Point out that Bob and Charlie deliver [Chat #3], while Alice prints [Demo Simulation] ! Artificially dropped incoming packet seq=3.
    - Have Charlie send "Message Four" (#4). Point out Alice's terminal:
        1. She receives seq=4 while expecting 3.
        2. She buffers seq=4 in her **Hold-Back Queue** instead of printing it out of order.
        3. She automatically sends a NACK for seq=3 to the Leader.

    4. The Leader retransmits seq=3, and Alice delivers [Chat #3] ... (recovered via NACK) followed immediately by [Chat #4] from her Hold-Back Queue!

4. **Demonstrate Fault Tolerance & Leader Crash Recovery (fault_tolerance.py)**:
    - Press Ctrl+C on Charlie (Node 300, the current Leader) to simulate a sudden Leader crash.
    - Wait **6 seconds** (3 missed heartbeats).
    - Point out that Alice and Bob print [Fault Tolerance] ! Elected Leader crashed! Triggering Bully Leader Election..., remove Node 300 from their Group View, elect Bob (Node 200) as the new Leader, and seamlessly continue sequencing the next chat message at [Chat #5]!